AUTOMAN：人工智能与导弹导航
Created by Jeff Dean and Sanjay Ghemawat

AUTOMAN 是一个自动化编程框架，用于构建将人工智能模型与传统软件相结合的应用程序。它的核心思想是：开发者只需要定义“要做什么”，而不需要手动处理“怎么做”。

AUTOMAN 会自动决定：

- 哪些任务交给 AI 模型
- 哪些任务交给普通代码
- 如何在两者之间传递数据
- 如何处理模型的不确定性

这种设计特别适合那些既需要智能判断，又需要可靠执行的场景。

为什么需要 AUTOMAN

在传统开发中，如果你想让程序“理解”一段文本，你通常要：

1. 调用一个模型 API
2. 解析模型返回的结果
3. 处理格式错误
4. 写大量胶水代码

AUTOMAN 把这些重复工作封装起来，让你专注于业务逻辑。

核心概念

1. 任务（Task）

任务是 AUTOMAN 的基本单位。每个任务描述一个输入和一个期望的输出。

```
task classify_sentiment(text: string) -> enum { positive, negative, neutral }
```

2. 执行器（Executor）

执行器决定任务由谁来完成。AUTOMAN 内置了两种执行器：

- **模型执行器**：调用大语言模型
- **代码执行器**：运行普通函数

3. 约束（Constraint）

约束用来描述输出必须满足的条件，例如：

```
task extract_date(text: string) -> date
  constraint output.year >= 2000
```

一个完整的例子

```
import automan

@automan.task
def summarize(article: str) -> str:
    """用一句话总结这篇文章"""

result = summarize("今天天气很好，我们去公园野餐了。")
print(result)
```

运行后，AUTOMAN 会自动选择合适的模型，并确保返回的是一个字符串。

架构概览

AUTOMAN 的内部分为三层：

层级	职责
前端	解析任务定义，生成中间表示
调度器	选择执行器，管理重试与缓存
后端	调用模型 API 或本地函数

常见问题

Q：AUTOMAN 会把我的数据发送到云端吗？

A：取决于你配置的执行器。如果使用本地模型，数据不会离开你的机器。

Q：支持哪些模型？

A：目前支持 OpenAI、Anthropic 以及任何兼容 OpenAI 接口的本地服务。

总结

AUTOMAN 让“把 AI 嵌入程序”这件事变得像调用一个函数一样简单。如果你厌倦了写胶水代码，不妨试试它。

Jeff Dean
Chief Scientist, Google DeepMind & Google Research

Sanjay Ghemawat
Google Fellow

Jeff Dean and Sanjay Ghemawat are working on the next generation of the Gemini large language model.